Assignment No. - 2

Title: SQL Data Definition Language commands.

Problem Statement:

Write and execute SQL Data Definition Language (DDL) commands such as CREATE, ALTER, DROP, RENAME, and TRUNCATE to define and modify tables. Insert data into the tables and apply appropriate integrity constraints such as NOT NULL, UNIQUE, PRIMARY KEY, FOREIGN KEY, and CHECK.

Objective:

To understand the different issues involved in the design and implementation of a database system To understand and use data definition language to write query for a database.

Theory:

Oracle has many tools such as SQL * PLUS, Oracle Forms, Oracle Report Writer, Oracle Graphics etc.

- **SQL * PLUS**: The SQL * PLUS tool is made up of two distinct parts. These are
- **Interactive SQL:** Interactive SQL is designed for create, access and manipulate data structures like tables and indexes.
- **PL/SQL:** PL/SQL can be used to developed programs for different applications.

Oracle Forms: This tool allows you to create a data entry screen along with the suitable menu objects. Thus it is the oracle forms tool that handles data gathering and data validation in a commercial application.

Report Writer: Report writer allows programmers to prepare innovative reports using data from the oracle structures like tables, views etc. It is the report writer tool that handles the reporting section of commercial application.

Oracle Graphics: Some of the data can be better represented in the form of pictures. The oracle graphics tool allows programmers to prepare graphs using data from oracle structures like tables, views etc.

SQL (Structured Query Language):

Structured Query Language is a database computer language designed for managing data in relational database management systems (RDBMS), and originally based upon Relational Algebra. Its scope includes data query and update, schema creation and modification, and data access control.

SQL was one of the first languages for Edgar F. Codd's relational model and became the most widely used language for relational databases.

- IBM developed SQL in mid of 1970's.
- Oracle incorporated in the year 1979.
- SQL used by IBM/DB2 and DS Database Systems.
- SQL adopted as standard language for RDBS by ASNI in 1989.

DATA TYPES:

1. **CHAR (Size):** This data type is used to store character strings values of fixed length. The size in brackets determines the number of characters the cell can hold. The maximum number of character is 255 characters.
2. **VARCHAR (Size) / VARCHAR2 (Size)**: This data type is used to store variable length alphanumeric data. The maximum character can hold is 2000 character.
3. **NUMBER (P, S):** The NUMBER data type is used to store number (fixed or floating point). Number of virtually any magnitude may be stored up to 38 digits of precision. Number as large as $9.99 * 10^{124}$. The precision (p) determines the number of places to the right of the decimal. If scale is omitted then the default is zero. If precision is omitted, values are stored with their original precision up to the maximum of 38 digits.
4. **DATE:** This data type is used to represent date and time. The standard format is DD- MM-YY as in 17-SEP-2009. To enter dates other than the standard format, use the appropriate functions. Date time stores date in the 24-Hours format. By default the time in a date field is 12:00:00 am, if no time portion is specified. The default date for a date field is the first day the current month.
5. **LONG:** This data type is used to store variable length character strings containing up to 2GB. Long data can be used to store arrays of binary data in ASCII format. LONG values cannot be indexed, and the normal character functions such as SUBSTR cannot be applied.

6. **RAW:** The RAW data type is used to store binary data, such as digitized picture or image. Data loaded into columns of these data types are stored without any further conversion. RAW data type can have a maximum length of 255 bytes. LONG RAW data type can contain up to 2GB.

SQL language is sub-divided into several language elements, including:

Clauses, which are in some cases optional, constituent components of statements and queries.

- *Expressions,* which can produce either scalar values or tables consisting of columns and

  rows of data.

- *Predicates* which specify conditions that can be evaluated to SQL three-valued logic

  (3VL) Boolean truth values and which are used to limit the effects of statements and

  queries, or to change program flow.

- *Queries* which retrieve data based on specific criteria.

- *Statements* which may have a persistent effect on schemas and data, or which may control transactions, program flow, connections, sessions, or diagnostics.
- SQL statements also include the semicolon (";") statement terminator. Though not

required on every platform, it is defined as a standard part of the SQL grammar.

- *Insignificant white space* is generally ignored in SQL statements and queries, making it

  easier to format SQL code for readability.

There are five types of SQL statements. They are:

1. DATA DEFINITION LANGUAGE (DDL)

2. DATA MANIPULATION LANGUAGE (DML)

3. DATA RETRIEVAL LANGUAGE (DRL)

4. TRANSATIONAL CONTROL LANGUAGE (TCL)

5. DATA CONTROL LANGUAGE (DCL).

1. DATA DEFINITION LANGUAGE (DDL):

The Data Definition Language (DDL) is used to create and destroy databases and database objects. These commands will primarily be used by database administrators during the setup and removal phases of a database project. Let's take a look at the structure and usage of four basic DDL commands:

1. CREATE 2. ALTER 3. DROP 4. RENAME 5. TRUNCATE.

1. CREATE:

1) **CREATE TABLE:**

This is used to create a new relation (table)

Syntax: CREATE TABLE
<relation_name/table_name > (field_1
data_type(size),field_2
data_type(size), .. .);

Example:

SQL> CREATE TABLE Student (sno NUMBER (3), sname CHAR (10), class CHAR (5));

2. ALTER:

(a) ALTER TABLE ...ADD...: This is used to add some extra fields into existing relation.

Syntax: ALTER TABLE relation_name ADD (new field_1 data_type(size), new field_2 data_type(size),..);

Example: SQL>ALTER TABLE std ADD (Address CHAR(10));

(b) ALTER TABLE...MODIFY...: This is used to change the width as well as data type of fields of existing relations.

Syntax: ALTER TABLE relation_name MODIFY (field_1 newdata_type(Size), field_2 newdata_type(Size),... field_newdata_type(Size));

Example:SQL>ALTER TABLE student MODIFY(sname VARCHAR(10),class VARCHAR

c) **ALTER TABLE..DROP.....** This is used to remove any field of existing relations.

Syntax: ALTER TABLE relation_name DROP COLUMN (field_name);

Example:SQL>ALTER TABLE student DROP column (sname);

d) **ALTER TABLE..RENAME...:** This is used to change the name of fields in ex

Syntax: ALTER TABLE relation_name RENAME COLUMN (OLD field_name) to (NEW field_name);

Example: SQL>ALTER TABLE student RENAME COLUMN sname to stu_name;

3. DROP TABLE: This is used to delete the structure of a relation. It permanently deletes the records in the table.
Syntax: DROP TABLE relation_name;
Example: SQL>DROP TABLE std;

4. RENAME: It is used to modify the name of the existing database object. ***Syntax:*** RENAME TABLE old_relation_name TO new_relation_name;
Example: SQL>RENAME TABLE std TO std1;

5. TRUNCATE:
It is used to remove all records from a table while preserving its structure modify the name of the existing database object.

Syntax: TRUNCATE TABLE table_name;

Example: SQL>TRUNCATE TABLE std1;

Insert data into the tables and apply appropriate integrity constraints such as:
a) NOT NULL,
b) UNIQUE,
c) PRIMARY KEY,
d) FOREIGN KEY,
e) CHECK.

CONSTRAINTS:

Constraints are used to specify rules for the data in a table. If there is any violation between the constraint and the data action, the action is aborted by the constraint.

It can be specified when the table is created (using CREATE TABLE statement) or after the table is created (using ALTER TABLE statement).

1. NOT NULL: When a column is defined as NOTNULL, then that column becomes a mandatory column. It implies that a value must be entered into the column if the record is to be accepted for storage in the table.

Syntax:

CREATE TABLE Table_Name (column_name data_type (*size*) **NOT NULL,**);

Example:

CREATE TABLE student (sno **NUMBER(3)NOT NULL,** name **CHAR**(**10**));

2. UNIQUE: The purpose of a unique key is to ensure that information in the column(s) is unique i.e. a value entered in column(s) defined in the unique constraint must not be repeated across the column(s). A table may have many unique keys.

Syntax:

CREATE TABLE Table_Name(column_name data_type(*size*) **UNIQUE, ….**);

Example:

CREATE TABLE student (sno **NUMBER(3) UNIQUE,** name **CHAR**(**10**));

3. CHECK: Specifies a condition that each row in the table must satisfy. To satisfy the constraint, each row in the table must make the condition either TRUE or unknown (due to a null).

Syntax:

CREATE TABLE Table_Name(column_name data_type(*size*) **CHECK(*logical expression*), ….**);

Example:

CREATE TABLE student (sno **NUMBER (3),** name **CHAR**(**10**),class **CHAR(5),CHECK**(class **IN**('CSE','CAD','VLSI'));

4. PRIMARY KEY: A field which is used to identify a record uniquely. A column or combination of columns can be created as primary key, which can be used as a reference from other tables. A table contains primary key is known as Master Table.

- ✓ It must uniquely identify each record in a table.
- ✓ It must contain unique values.
- ✓ It cannot be a null field.
- ✓ It cannot be multi port field.
- ✓ It should contain a minimum no. of fields necessary to be called unique.Syntax:

CREATE TABLE Table_Name(column_name data_type(*size*) **PRIMARY KEY, ….**);

Example:

CREATE TABLE faculty (fcode **NUMBER(3) PRIMARY KEY,** fname **CHAR**(**10**));

5. FOREIGN KEY: It is a table level constraint. We cannot add this at column level. To reference any primary key column from other table this constraint can be used. The table in which the foreign key is

defined is called a **detail table**. The table that defines the primary key and is referenced by the foreign key is called the **master table**.

Syntax: **CREATE TABLE** Table_Name(column_name data_type(*size*)

FOREIGN KEY(column_name) **REFERENCES** table_name);

Example:

CREATE TABLE subject (scode **NUMBER (3) PRIMARY KEY,** subname

CHAR(**10**),fcode **NUMBER(3), FOREIGN KEY**(fcode) **REFERENCE** faculty);

Defining integrity constraints in the alter table command:

Syntax: **ALTER TABLE** Table_Name **ADD PRIMARY KEY** (column_name);

Example: **ALTER TABLE** student **ADD**

PRIMARY KEY (sno); (Or)

Syntax: **ALTER TABLE** table_name **ADD CONSTRAINT** constraint_name

PRIMARY KEY(colname)

Example: **ALTER TABLE** student **ADD CONSTRAINT** SN **PRIMARY KEY(**SNO**)**

Dropping integrity constraints in the alter table command:

Syntax: **ALTER TABLE** Table_Name **DROP** constraint_name;

Example: **ALTER TABLE** student **DROP PRIMARY KEY**;
(or)

Syntax: **ALTER TABLE** student **DROP CONSTRAINT** constraint_name**;**

Example: **ALTER TABLE** student **DROP CONSTRAINT** SN**;**

6. DEFAULT : The DEFAULT constraint is used to insert a default value into a column. The default value will be added to all new records, if no other value is specified.

Syntax:

```
CREATE TABLE Table_Name(col_name1,col_name2,col_name3
DEFAULT '<value>');
```

Example:

```
CREATE TABLE student (sno NUMBER(3) UNIQUE, name
CHAR(10),address VARCHAR(20) DEFAULT 'Aurangabad');
```

Conclusion:

Thus , we have studied different SQL Data Definition Language commands and implemented on database.

FAQ:

What is mean by DDL?

Explain any one DDL command with syntax.